ZERO TO MASTERY / LOCAL LLM LAB

LoRA Fine-tune Studio: Zero to Mastery

NLP, transformers, parameter-efficient fine-tuning, preference optimization, evaluation, and this repository

22 chapters · 2 guided labs · 1 capstone

Contents

Module 1 - Learn the map before the territory

Objectives

By the end of this module, you can explain what this application produces, identify the stages of an LLM project, and choose an appropriate path through the course.

The result is an adapter, not a new foundation model

LoRA Fine-tune Studio starts with an existing causal language model from Hugging Face. It freezes the base model and trains a small set of adapter parameters. A completed run therefore produces two things that must be used together:

1. the original base model at a compatible revision; and
2. the trained PEFT adapter and tokenizer files.

The application supports five post-training approaches and four adapter methods:

Dimension	Implemented choices
Approach - what objective is learned	Supervised Fine-Tuning, Reward Modeling, DPO, KTO, ORPO
Method - how parameters are adapted	LoRA, QLoRA, OFT, QOFT
Backend - which runtime executes	Standard Transformers/PEFT/TRL; optional Windows Unsloth for LoRA/QLoRA

Training runs locally on one NVIDIA CUDA GPU. GitHub hosts source code; Hugging Face can host models, datasets, and optional adapter uploads; neither service supplies local training compute.

The end-to-end learning loop

```
Define task -> establish baseline -> collect examples -> validate data
    -> choose model and recipe -> smoke test -> train -> evaluate
    -> diagnose errors -> iterate one variable -> document and deploy responsibly
```

The visible training step is only one part of this loop. Data quality, evaluation, and deployment constraints usually determine whether the project is valuable.

A vocabulary preview

- **NLP** studies computational methods for human language.
- **Model** is a parameterized function that maps inputs to predictions.
- **Parameter** is a learned numeric value, commonly a matrix element.
- **Token** is the model's discrete unit of text.
- **Embedding** is a learned vector representation.
- **Transformer** is the neural architecture behind the models targeted here.
- **Loss** is a scalar training objective to minimize.
- **Gradient** describes how a small parameter change affects loss.
- **Fine-tuning** continues training from pretrained weights on a narrower objective.
- **Adapter** is a small trainable module attached to a frozen base model.
- **Inference** uses trained weights to produce predictions or text.

Choose your path

Goal	Recommended modules
Inspect the workflow without a GPU	Run `demo/streamlit_app.py`, then modules 1, 11, and 15
Run a safe first experiment	1, 2, 5, 8, 10, 11, 12, 15
Understand transformer mechanics	2, 3, 4, 5, 7
Build a preference-training project	2-10, 13-15, 17
Maintain or extend this repository	All modules, especially 9, 10, 16

Checkpoint

1. Why is an adapter not a standalone model?
2. Which choices describe the objective, and which describe parameter adaptation?
3. Why should evaluation be designed before a long training run?

Answers appear in Appendix A.

Module 2 - Natural language becomes model input

Objectives

You will understand common NLP tasks, corpora, tokens, vocabulary, context windows, tokenization, and why this application does not require a separate embedding model.

What NLP systems do

Natural language processing covers tasks such as classification, information extraction, search, translation, summarization, question answering, and open-ended generation. A causal language model reduces many of these tasks to one operation: predict the next token given all previous tokens.

For tokens `x_1 ... x_T`, the model represents a sequence probability as:

```
P(x_1, ..., x_T) = product from t=1 to T of P(x_t | x_1, ..., x_(t-1))
```

Generation repeatedly samples or selects from the next-token distribution, appends the new token, and runs again until a stopping condition.

Corpus, example, field, and label

A **corpus** is a collection of language data. A **training example** is one unit shown to the optimizer. Examples contain **fields** such as `prompt`, `completion`, `chosen`, or `rejected`. A **label** is the desired prediction target. In causal language modeling, the next tokens themselves act as labels; in Reward Modeling, the objective learns which response in a pair should receive the higher score.

Tokenization

Models do not consume raw strings. A tokenizer maps text to integer token IDs using a fixed vocabulary. Tokens may represent a whole word, part of a word, punctuation, whitespace patterns, or bytes. Therefore:

- one word is not necessarily one token;
- token counts vary by tokenizer and language;
- unusual formatting and code can tokenize differently from prose; and
- character length is not a reliable estimate of GPU sequence length.

Special tokens can mark sequence boundaries, padding, roles, or control instructions. Chat models also use a **chat template** that converts structured messages into the exact tokenizable text the model expects. In this repository, conversational SFT data is formatted with the selected tokenizer's chat template.

Vocabulary and context

The **vocabulary** is the finite token-to-ID mapping. The **context window** is the maximum number of tokens the architecture and runtime can consider in one sequence. The app's `max_length` is a training cap from 128 to 8192; it does not expand a model's architectural context window. A chosen value must fit both the model and the GPU.

Longer sequences cost more memory and computation because activations grow with token count and self-attention compares token positions. Packing more useful examples into a shorter clean format is often better than blindly increasing maximum length.

Embeddings: two meanings people confuse

An **embedding layer** inside the language model maps each token ID to a dense vector. If the vocabulary has `V` tokens and the hidden size is `d`, the embedding table has shape `V x d`. Every model in this application already contains such an embedding layer.

An **embedding model** commonly means a separate model that converts a sentence or document into one vector for search, clustering, or retrieval-augmented generation. This fine-tuning application does not perform semantic search or RAG, so it does not need a separate embedding model. You would add one in a retrieval system outside this training pipeline.

Data is behavior specification

Examples teach patterns, not intentions hidden in your head. If desired answers are concise but the dataset contains verbose answers, the model learns verbosity. If role labels are inconsistent, the model learns inconsistent turn structure. If private or copyrighted text is included without authorization, training does not remove that risk.

Mini exercise: estimate tokens

Take three examples from `examples/sft_sample.jsonl` and predict which will produce the most tokens. Later, inspect them with the selected model tokenizer. Record why your character-based prediction was right or wrong.

Checkpoint

1. What is the difference between a token and a word?
2. Why does this app need token embeddings but not a separate embedding model?
3. What can go wrong when the dataset's chat format differs from the base model's expected template?

Module 3 - Neural networks learn with vectors and gradients

Objectives

You will understand tensors, parameters, forward passes, losses, gradients, optimizers, learning rate, batches, epochs, and the memory used during training.

Tensors and linear layers

A tensor is a multidimensional array. Language models use tensors for token IDs, embeddings, activations, weights, gradients, and optimizer state. A linear layer maps an input vector `x` to an output vector:

```
y = W x + b
```

`W` and `b` are parameters learned during training. Transformer models contain many linear projections arranged into repeated blocks.

Forward pass and loss

The **forward pass** applies the current parameters to a batch and produces logits. Logits are unnormalized scores for each vocabulary token. Softmax converts them into a probability distribution:

```
softmax(z_i) = exp(z_i) / sum_j exp(z_j)
```

For a target token `y`, cross-entropy loss is the negative log probability assigned to that target:

```
L = -log P(y | context)
```

A lower training loss means the model predicts training targets with greater confidence. It does not automatically mean outputs are factually correct, safe, or useful on unseen inputs.

Backpropagation and gradient descent

Backpropagation applies the chain rule to compute a gradient for each trainable parameter. A basic update is:

```
theta_(t+1) = theta_t - learning_rate * gradient(L(theta_t))
```

Real trainers use optimizers such as AdamW, which maintain additional statistics and adapt updates. The learning rate still controls update scale:

- too small can make learning impractically slow;
- too large can cause unstable loss or destroy useful behavior; and
- the best scale depends on objective, model, adapter, batch, and data.

The app therefore uses different defaults: `2e-4` for SFT, `1e-3` for Reward Modeling, and `1e-5` for DPO, KTO, and ORPO.

Batches, accumulation, steps, and epochs

The **per-device batch size** is the number of examples processed together in one forward/backward pass on the GPU. **Gradient accumulation** delays the optimizer update and adds gradients across several small batches.

```
effective batch size = per-device batch size * accumulation steps * number of devices
```

This app uses one device, so batch size 1 with accumulation 8 behaves like an effective batch of 8 for optimizer-update frequency, while usually needing far less peak VRAM than processing 8 examples simultaneously.

A **step** usually means one optimizer update. An **epoch** is one pass through the selected training rows. The approximate number of optimizer steps is:

```
steps per epoch ~= ceil(number of training examples / effective batch size)
total steps ~= steps per epoch * epochs
```

Dynamic padding, distributed behavior, and trainer details can modify the exact count. The Smoke preset uses an explicit maximum of 20 steps, which takes priority over completing all requested epochs.

Gradient clipping and checkpointing

The gradient norm summarizes gradient magnitude. **Gradient clipping** rescales overly large gradients to reduce unstable updates. `max_grad_norm=1.0` is the app default; zero disables clipping.

Gradient checkpointing saves memory by discarding selected forward activations and recomputing them during backpropagation. It trades additional computation for lower activation memory. This is different from a disk **training checkpoint**, which saves adapter, optimizer, scheduler, RNG, and trainer state for recovery.

Where GPU memory goes

Training can allocate memory for:

- frozen base weights;
- trainable adapter weights and their gradients;
- optimizer state for trainable weights;
- forward activations;
- temporary kernels and quantization metadata; and
- CUDA allocator reservations.

QLoRA reduces the largest component - frozen base weights - but does not remove the other components. Sequence length and batch size can still cause out-of-memory errors.

Checkpoint

1. What is the difference between an optimizer step and an epoch?
2. Why can accumulation reduce peak memory without changing effective batch size?
3. Why does QLoRA not guarantee that any model fits a particular GPU?

Module 4 - Build a transformer from attention blocks

Objectives

You will follow a token through a decoder-only transformer and understand attention, causal masking, multi-head projections, residual streams, normalization, MLPs, and generation.

From token IDs to hidden states

For a batch size `B`, sequence length `T`, and hidden dimension `d`, token IDs have shape `B x T`. Embedding lookup produces hidden states with shape `B x T x d`. Positional information is added or applied so the model can distinguish token order.

Each transformer block reads and updates a **residual stream**. A simplified decoder block contains:

1. normalization;
2. masked multi-head self-attention;
3. a residual addition;
4. another normalization;
5. a position-wise feed-forward network; and
6. another residual addition.

Scaled dot-product attention

Linear projections turn hidden states `X` into queries, keys, and values:

```
Q = X W_Q
K = X W_K
V = X W_V
Attention(Q, K, V) = softmax((Q K^T) / sqrt(d_k) + mask) V
```

Interpretation:

- a query describes what the current position is looking for;
- a key describes what each earlier position offers;
- their dot product produces a compatibility score;
- softmax converts scores to weights; and
- the weighted values carry information into the current position.

Dividing by `sqrt(d_k)` prevents dot products from growing so large that softmax becomes extremely sharp early in training.

Causal masking

A causal language model must not see future target tokens. The mask adds a very negative value to forbidden attention scores above the diagonal. Position `t` can attend only to positions `1..t`. This matches next-token prediction and autoregressive generation.

Multi-head attention

Instead of one large attention calculation, the model divides projections into heads. Different heads can learn different relational patterns. Their outputs are concatenated and projected through `W_O`. Modern model implementations may vary the number of query, key, and value heads, but the query-key-value mental model remains useful.

The feed-forward network

The MLP processes each sequence position independently with learned linear layers and a nonlinearity or gating function. Llama-like models commonly expose projections named `gate_proj`, `up_proj`, and `down_proj`. Attention projections commonly include `q_proj`, `k_proj`, `v_proj`, and `o_proj`. The Unsloth LoRA path in this repository targets exactly those seven projection families.

Residuals and normalization

Residual connections let each block add a refinement rather than rebuild the entire representation. Normalization keeps activation scales manageable. These design choices help very deep networks optimize and preserve information across layers.

Logits and decoding

After the final block, a language-model head maps the last hidden representation to one logit per vocabulary token. Decoding chooses the next token. Common policies include:

- greedy decoding: select the highest-probability token;
- temperature sampling: flatten or sharpen probabilities;
- top-k/top-p sampling: restrict candidate tokens; and
- beam search: maintain multiple high-scoring sequences.

The app's base-versus-adapter comparison uses deterministic generation with `do_sample=False`, so it isolates adapter effects better than a random sampling test.

Why adapter targets matter

Attention and MLP projections contain much of the block's learned transformation. LoRA and OFT modify how these projections behave without training every base parameter. The choice of target modules controls adapter capacity, memory, and compatibility.

Exercise: trace one token

Draw the path of the final input token from token ID through embedding, one attention block, one MLP, the language-model head, softmax, and next-token selection. Label every matrix that an all-linear LoRA configuration can adapt.

Checkpoint

1. Why is the causal mask required during training even when the full target sequence is present in memory?
2. What roles do Q, K, and V play?
3. Why are attention and MLP projections useful adapter targets?

Module 5 - Understand the LLM lifecycle

Objectives

You will distinguish pre-training, instruction tuning, preference optimization, inference, prompting, RAG, and fine-tuning, then choose the cheapest suitable technique.

Pre-training

Pre-training learns broad language and world patterns from enormous corpora, usually with next-token prediction. It requires much more data and compute than this local app is designed to provide. Continued pre-training applies the same broad objective to a domain corpus, but it still differs from teaching a precise input-output behavior.

Pre-training is **context only** in this repository.

Supervised instruction tuning

Instruction tuning presents desired behavior: a prompt or conversation and the answer the model should produce. SFT makes that answer more probable. It is the most direct starting point when you can write high-quality demonstrations.

Preference alignment

Preference data compares candidate responses. One path trains a reward model, then uses reinforcement learning such as PPO to optimize a policy. Direct objectives such as DPO avoid a separate online RL loop and update the policy from preference pairs.

This app implements Reward Modeling, DPO, KTO, and ORPO. It does not implement PPO.

Inference and serving

Inference loads a base model, optionally attaches an adapter, and generates or scores inputs. Serving adds operational concerns: batching, latency, caching, concurrency, observability, authentication, and deployment hardware. This repository provides only on-demand local comparison, not a production serving system.

Prompting, RAG, or fine-tuning?

Use the least expensive technique that solves the measured problem:

Need	First technique to try	Reason
Better instructions or output format	Prompting	No training or artifact lifecycle
Current/private factual knowledge	RAG	Retrieves changing facts without encoding them in weights
Stable style, schema, or task behavior	SFT/PEFT	Learns repeated behavior from demonstrations
Choose consistently preferred behavior	Preference optimization	Learns comparative quality signals
Broad new language/domain knowledge	Continued pre-training	Requires a different scale and pipeline

Fine-tuning is a poor substitute for a database. It cannot guarantee exact recall, freshness, attribution, or removal of one fact. RAG and tools are usually better for those requirements.

Baseline before adaptation

Write a fixed evaluation set and test the unmodified model with your best prompt. A fine-tuned model should beat that baseline on relevant held-out cases without causing unacceptable regressions. Otherwise, the extra data, compute, and deployment complexity has not earned its place.

Checkpoint

1. When is RAG a better fit than fine-tuning?
2. How does SFT data differ from preference data?
3. Why is pre-training outside this application's practical scope?

Module 6 - Choose a tuning family

Objectives

You will compare full tuning, freeze tuning, and PEFT, then explain why adapters fit local single-GPU experimentation.

Full tuning

Full tuning updates all base-model parameters. It provides maximum flexibility but requires gradients and optimizer state for every trainable weight. It also creates a full model artifact for every task. On an 8 GB RTX 4060, full tuning modern LLMs is usually impractical.

Full tuning is **context only** here.

Freeze tuning

Freeze tuning updates selected existing layers while leaving the rest frozen. It can reduce cost, but the selected full layers may still contain many parameters and each task still modifies part of the base model directly.

Freeze tuning is **context only** here.

Parameter-efficient fine-tuning

PEFT freezes original weights and adds or selects a small trainable parameter set. Its advantages include:

- fewer trainable parameters and optimizer states;
- smaller task-specific artifacts;
- multiple adapters can share one base model; and
- the original model remains available for comparison.

PEFT does not eliminate the need to load the base model. Base-weight precision and activations still dominate memory for many workloads.

Adapter lifecycle

```
Base model + adapter configuration -> train adapter -> save adapter
Base model + saved adapter -> inference or evaluation
```

An adapter configuration records the base-model identifier, PEFT type, target modules, rank or transformation settings, and other compatibility data. Keep it with the adapter weights and training configuration.

Method and objective are independent axes

SFT does not mean LoRA, and QLoRA does not mean DPO. The objective defines the loss and dataset; the adapter method defines trainable parameterization. The app's recipe table allows every implemented objective with LoRA, QLoRA, OFT, or QOFT.

Decision table

Constraint	Starting choice
6-10 GB VRAM	QLoRA, small model, Smoke preset
More VRAM and avoid 4-bit base loading	LoRA
Explore orthogonal transformations	OFT
Need OFT with reduced base-weight memory	QOFT

Constraint	Starting choice
Need native Windows acceleration	Unsloth with LoRA/QLoRA
Need maximum base-weight flexibility	Different full-tuning system

Checkpoint

1. Which memory components remain even with PEFT?
2. Why can many adapters share one base model?
3. What is the difference between a method and an objective?

Module 7 - Master LoRA, QLoRA, OFT, and QOFT

Objectives

You will understand the mathematics and implementation tradeoffs of all four methods available in the app.

LoRA

For a frozen linear weight `W` with shape `d_out x d_in`, LoRA learns two smaller matrices:

```
A has shape r x d_in
B has shape d_out x r
W' x = W x + scaling * B A x
scaling = alpha / r
```

The dense update would have `d_out * d_in` parameters. LoRA uses approximately:

```
r * (d_in + d_out)
```

When rank `r` is much smaller than either dimension, the reduction is large. Higher rank increases capacity and cost. `alpha` controls update scale, target modules select where updates are inserted, and dropout regularizes the adapter branch.

The standard backend in this repository uses rank 16, alpha 32, dropout 0.05, no bias, and `target_modules="all-linear"`.

QLoRA

QLoRA keeps LoRA adapters trainable in a floating-point compute type while loading the frozen base weights in four-bit representation. This repository uses:

- NF4 quantization;
- double quantization; and
- BF16, FP16, or FP32 computation on the standard backend.

NF4 is designed for normally distributed neural weights. Double quantization also quantizes quantization constants to save additional memory. Computation cannot happen directly as arbitrary four-bit arithmetic; values are dequantized into the configured compute dtype for operations.

QLoRA's main benefit is memory, not a guarantee of better output quality. Quantization introduces approximation error, while the reduced memory can make otherwise impossible experiments practical.

OFT

Orthogonal Fine-Tuning learns transformations constrained to be orthogonal. An orthogonal matrix `R` satisfies:

```
R^T R = I
```

Orthogonal transformations preserve vector norms and angles. OFT uses structured parameterization so it can learn useful rotations with fewer parameters than updating the full dense matrix. This repository configures block size 32, Cayley-Neumann transformations, all-linear targets, and no bias.

QOFT

QOFT combines four-bit frozen base weights with OFT adapters. Its memory relationship to OFT is analogous to QLoRA's relationship to LoRA. The current implementation applies a narrow compatibility bridge for the installed PEFT version's four-bit OFT dispatcher.

Standard versus Unsloth settings

Method	Standard backend	Unsloth backend
LoRA	All linear targets; rank 16; alpha 32; dropout 0.05	Seven attention/MLP projections; rank 16; alpha 32; dropout 0
QLoRA	Standard LoRA plus NF4 base	Unsloth four-bit loading plus optimized adapter path
OFT	Supported	Not supported by this integration
QOFT	Supported with compatibility bridge	Not supported by this integration

Unsloth is not a new learning objective. It changes execution kernels, loading, adapter injection, optimizer choice, and checkpointing behavior while preserving the selected approach.

Compute type resolution

UI request	Effective type
Auto	BF16 when the GPU supports it; otherwise FP16
BF16	BF16 when supported; otherwise FP16
FP16	FP16
FP32	FP32; standard backend only

BF16 has FP32-like exponent range with lower mantissa precision, which often makes it more stable than FP16 on supported hardware. FP32 greatly increases memory and is not automatically higher quality for adapter training.

Exercise: count a LoRA layer

For a `4096 x 4096` linear weight and rank 16:

1. calculate dense parameter count;
2. calculate LoRA parameter count;
3. calculate the percentage that LoRA trains; and
4. explain why total GPU memory does not fall by the same percentage.

Checkpoint

1. What does rank control?
2. What is quantized in QLoRA and what remains trainable?
3. What property does an orthogonal transform preserve?

Module 8 - Engineer datasets that teach the intended behavior

Objectives

You will create supported schemas, inspect and normalize data, combine sources, avoid leakage, and build a small but credible evaluation set.

Supported source boundaries

The app accepts:

- a Hugging Face dataset repository ID such as `owner/name`;
- a root HTTPS repository URL on `huggingface.co`; or
- an uploaded `.csv`, `.json`, or `.jsonl` file no larger than 200 MB.

Uploads are stored by a SHA-256 content hash under `.uploads/`, so the original filename cannot choose an arbitrary destination path.

Canonical SFT shapes

Plain text:

```
{"text": "A complete piece of training text."}
```

Prompt and completion:

```
{"prompt": "Explain gradient accumulation.", "completion": "It combines gradients across several small batches before
```

Conversation:

```
{"messages": [{"role": "user", "content": "What is QLoRA?"}, {"role": "assistant", "content": "QLoRA trains LoRA adapt
```

Canonical preference shape

```
{"prompt": "Give one safe first-run recommendation.", "chosen": "Use a small model, QLoRA, and the Smoke preset.", "re
```

Reward Modeling, DPO, KTO, and ORPO all use this three-column contract in the current app. The trainers interpret the data differently; shared columns do not make their objectives identical.

Inspection and mapping

Detection checks for preference columns first, then `messages`, `text`, and prompt/completion. Other schemas require manual mapping. The saved `DatasetSpec` records source, split, format, and column roles. The worker reloads the source and reduces it to canonical columns before training.

Structural validation cannot judge whether an answer is correct, whether a preference is meaningful, or whether examples are duplicated. Human and programmatic data review are still required.

Multiple datasets

Every selected source must normalize to the same canonical format. The worker:

1. loads and normalizes each source independently;
2. concatenates them in collection order;
3. shuffles the combined rows with the configured seed when multiple sources exist;
4. applies `max_samples` as one global cap; and
5. creates a seeded evaluation split when enabled and at least ten rows remain.

Sources are not balanced. A source with 900 rows contributes nine times as many rows as a source with 100 rows. If balance matters, curate or resample before adding the files.

Quality dimensions

Evaluate a dataset for:

- **correctness:** answers and preference labels are defensible;
- **relevance:** examples match the deployment task;
- **coverage:** important inputs and edge cases appear;
- **consistency:** tone, structure, and policy do not contradict;
- **diversity:** examples do not repeat one template mechanically;
- **legality and consent:** use is authorized;
- **privacy:** secrets and personal data are removed or properly governed;
- **separation:** validation/test cases and near-duplicates are excluded from training;
- **format integrity:** roles and columns match the chosen recipe; and
- **difficulty:** examples teach beyond what the baseline already does perfectly.

Leakage and memorization

If evaluation examples appear in training, the score measures recall rather than generalization. Near-duplicates can leak even when text is not byte-identical. Split by entity, conversation, document, or time when random row splitting would place closely related examples on both sides.

The app's automatic split is convenient for a smoke test, not a substitute for a carefully held-out external evaluation suite.

Data sizing

More data helps only when additional examples add signal. A few dozen clean examples can demonstrate pipeline correctness. A robust behavior change usually needs broader coverage and repeated evaluation. Start small, inspect failures, and add examples that address observed gaps rather than collecting volume blindly.

Lab preparation

Open `examples/sft_sample.jsonl` and `examples/preference_sample.jsonl`. For each file:

1. state the canonical format;
2. identify the behavior being taught;
3. identify one missing edge case; and
4. write one evaluation prompt that must not be added to training.

Checkpoint

1. Why must all combined datasets share a canonical format?
2. Does the app balance multiple datasets automatically?
3. Why is a random automatic split insufficient for a final product claim?

Module 9 - Select the post-training objective

Objectives

You will distinguish SFT, Reward Modeling, DPO, KTO, ORPO, and PPO and understand how their losses change model behavior.

Supervised Fine-Tuning

SFT minimizes token-level cross-entropy on desired text. Conceptually:

```
L_SFT = -sum over target tokens of log pi_theta(token | prior context)
```

It answers: "What should the model say for this input?" Use it when you can provide a good target response directly.

Reward Modeling

A reward model maps a prompt-response pair to a scalar score. For a chosen response `y_w` and rejected response `y_l`, a common pairwise objective encourages:

```
r_theta(x, y_w) > r_theta(x, y_l)
```

with a logistic loss such as:

```
L_RM = -log sigmoid(r_theta(x, y_w) - r_theta(x, y_l))
```

This repository loads a one-label sequence-classification model and preserves its `score` module in the adapter. The result scores responses; it is not directly a text generator, so Monitor does not offer base-versus-adapter generation for reward runs.

Direct Preference Optimization

DPO updates the policy from preference pairs without first training a separate reward model and running an online reinforcement-learning loop. It compares how the trainable policy and a reference policy favor chosen versus rejected responses. `beta` controls the strength of the preference/reference tradeoff.

Use DPO after SFT when the model can already respond but must choose consistently better behavior. Preference labels should reflect a clear rubric, not arbitrary taste.

KTO

KTO uses a prospect-theoretic preference objective designed to learn from desirable and undesirable outcomes. The current UI uses the shared preference-pair schema and requires per-device batch size at least 2. It also passes `beta` to the trainer.

ORPO

ORPO combines supervised likelihood and an odds-ratio preference penalty in one objective. It can teach the chosen response while separating it from the rejected one, without the same explicit reference-model arrangement as DPO. The app uses preference data and exposes `beta`.

PPO and the classic RLHF pipeline

Proximal Policy Optimization is an online reinforcement-learning algorithm. A classic RLHF pipeline can be summarized as:

```
SFT policy -> reward model -> generate samples -> score samples
    -> PPO policy updates constrained against a reference
```

This involves policy, reference, reward, and often value models plus rollout generation. It is substantially more complex and memory-intensive than the offline recipes in this app. PPO is **context only** here.

SimPO and other objectives

TRL and the research literature contain additional preference objectives such as SimPO, CPO, GRPO, and online DPO. Availability in an installed library is not enough to claim application support. A correct integration also needs a dataset contract, model path, controls, artifacts, tests, and documentation. These methods are **context only**.

Recipe table implemented by the app

Approach	TRL trainer	Data	Default LR	Beta	Minimum batch
SFT	`SFTTrainer`	messages/text/prompt-completion	`2e-4`	No	1
Reward	`RewardTrainer`	preference	`1e-3`	No	1
DPO	`DPOTrainer`	preference	`1e-5`	Yes	1
KTO	`KTOTrainer`	preference	`1e-5`	Yes	2
ORPO	`ORPOTrainer`	preference	`1e-5`	Yes	1

Objective selection

1. Start with SFT when desired answers can be demonstrated.
2. Establish an SFT baseline before preference tuning unless the base model already has adequate response behavior.
3. Use Reward Modeling when an explicit scorer is part of the system design.
4. Use DPO/KTO/ORPO when comparative judgments are easier or more reliable than writing one perfect answer.
5. Evaluate outputs, not just objective loss.

Checkpoint

1. Why can a Reward Modeling adapter not be evaluated with the current generation UI?
2. How does DPO differ operationally from reward-model-plus-PPO training?
3. What does `beta` influence?

Module 10 - Prepare the local training system

Objectives

You will understand the two runtimes, hardware scan, Hugging Face boundary, Unsloth integration, and conservative capacity planning.

Supported environment

The standard application supports native Windows 11 and x86-64 Linux with an NVIDIA GPU, CUDA-enabled PyTorch, and Python 3.14 in the uv-managed `.venv`.

Windows can also use `.venv-unsloth`, an isolated Python 3.13.13 environment pinned to the repository's Unsloth stack. The separation prevents incompatible Python/PyTorch requirements from contaminating the Streamlit runtime.

Use SETUP.md for installation commands. The conceptual dependency chain is:

```
NVIDIA driver -> CUDA-enabled PyTorch -> Transformers/Datasets
    -> PEFT + TRL + bitsandbytes -> optional Unsloth -> Streamlit app
```

What the System page checks

The read-only scan reports operating system, Python, CPU threads, available RAM, free disk, CUDA runtime, GPU, total/free VRAM, BF16 support, package versions, uv, Ollama, Unsloth, and whether a Hugging Face token is configured. It does not install drivers or display token values.

Conservative GPU recommendations

Total VRAM	App recommendation
Below 6 GB	QLoRA up to about 1B parameters
6 to below 10 GB	QLoRA up to about 3B
10 to below 16 GB	QLoRA up to about 7B
16 GB or more	QLoRA up to about 13B

The System page also looks for at least 3.5 GB currently free for the smallest QLoRA jobs. These thresholds are warnings, not proofs. Start with `Qwen/Qwen3-0.6B` and the Smoke preset on an 8 GB GPU.

Hugging Face access

Public repositories can work without a token. Gated/private downloads and Hub uploads require `HF_TOKEN` with suitable permissions. A token authenticates network access; it does not provide GPU compute or pay for training.

Tokens are read from the environment or ignored Streamlit secrets. They are not stored inside run configuration or training artifact JSON.

Standard backend versus Unsloth

Property	Standard	Repository Unsloth integration
Platform	Windows and Linux	Native Windows only
Methods	LoRA, QLoRA, OFT, QOFT	LoRA, QLoRA
Compute	Auto/BF16/FP16/FP32	Auto/BF16/FP16
Interpreter	Main `.venv`	`.venv-unsloth`
Model loading	Transformers	`FastLanguageModel`

Property	Standard	Repository Unsloth integration
Optimizer override	Trainer default	`adamw_8bit`
Dataset workers	Trainer default	1

The toggle becomes unavailable when the runtime check fails, OFT/QOFT is selected, or FP32 compute is selected. Unsloth support is also model-architecture dependent; a green runtime check cannot guarantee every model loads.

Before any lab

1. Open System and resolve runtime blockers.
2. Close unrelated GPU-heavy applications.
3. confirm adequate disk for model cache, checkpoints, and artifacts.
4. Verify gated-model access if applicable.
5. Keep the machine powered and avoid driver updates during training.

Checkpoint

1. Why does Unsloth use another Python environment?
2. Does a configured HF token mean training happens in the cloud?
3. Why are VRAM recommendations conservative rather than exact?

Module 11 - Translate intent into app settings

Objectives

You will navigate the eight pages and understand every training control and validation boundary.

The eight-page workflow

1. **System:** inspect readiness.
2. **Dataset:** load, preview, map, add, edit, remove, and order sources.
3. **Model:** validate repository/revision and inspect parameter count.
4. **GPU memory:** view global and process-local CUDA memory.
5. **Training:** select approach, method, backend, preset, and controls.
6. **Review & run:** inspect effective settings, then start or queue the run.
7. **Monitor:** inspect FIFO order, follow status/logs, cancel, resume, and evaluate.
8. **Ollama playground:** use models already installed in local Ollama.

Presets

Preset	Length	Epochs	Max steps	Max samples	Accumulation	Eval
Smoke test	512	1	20	100	4	Off
Standard	1024	2	Unlimited	All	8	On
Quality	2048	3	Unlimited	All	16	On

Use Smoke to validate the pipeline, not to claim final quality. Standard is a reasonable baseline. Quality increases cost and is not guaranteed to improve a small or noisy dataset.

Core controls

Approach and method are linked. The method dropdown contains only methods registered for the selected recipe. All current recipes expose all four standard methods.

Learning rate controls update scale. Default mode follows the recipe; Custom accepts `1e-7` through `1e-2`.

Epochs controls passes through the dataset unless `max_steps` stops first. Custom accepts values above zero through 20, including fractional epochs.

Maximum samples is a global cap applied after datasets are combined and shuffled. It is useful for smoke tests and controlled experiments.

Maximum gradient norm clips gradients above the threshold. The default is 1.0; zero disables clipping.

Compute type resolves Auto/BF16 to BF16 when supported and otherwise FP16. FP32 uses the standard backend.

Advanced controls

Maximum sequence length affects truncation/padding behavior, activation memory, and attention cost. Use the shortest value that preserves task-relevant information.

Beta controls preference/reference strength for DPO, KTO, and ORPO. Treat it as an objective-specific hyperparameter, not a generic quality slider.

Per-device batch size directly affects peak VRAM. KTO requires at least 2 in this app; other recipes default to 1.

Gradient accumulation increases effective batch size without placing all examples on the GPU at once.

Gradient checkpointing reduces activation memory through recomputation and should usually remain enabled on constrained GPUs.

Review-time blockers

Review prevents launch when configuration validation fails, CUDA is missing, a large model warning is unacknowledged, Unsloth is requested but unavailable, Hub publishing lacks a token, or another owned job is active.

The Review page displays requested and effective compute types. Read the entire JSON summary before starting because that saved configuration becomes the durable run contract.

One-variable experiments

Hold model, data, seed, and evaluation fixed. Change one important variable per run. Record the hypothesis before training. A useful order is:

1. prove the pipeline with Smoke;
2. establish Standard baseline;
3. change data quality or coverage;
4. tune learning rate or epochs;
5. tune length/batch only when the task requires it; and
6. compare adapter/backend methods only after evaluation is stable.

Checkpoint

1. Which limit wins in the Smoke preset: epochs or maximum steps?
2. Why is maximum samples applied after seeded shuffle?
3. Which settings most directly increase peak activation memory?

Module 12 - Lab A: complete an SFT smoke run

Goal

Validate the entire local pipeline with a small model and the included instructional dataset. This lab proves mechanics, not production quality.

Lab configuration

Setting	Value
Model	`Qwen/Qwen3-0.6B` at `main`
Dataset	`examples/sft_sample.jsonl`
Approach	Supervised Fine-Tuning
Method	QLoRA
Backend	Standard first; optional Unsloth comparison on Windows
Preset	Smoke test
Compute	Auto
Other controls	Defaults

Step 1: define a baseline

Before training, write five prompts about the dataset's target behavior. Do not copy training prompts. Save the exact wording and the base model's answers. Score each answer from 0-2 for correctness, relevance, and clarity.

Step 2: inspect the dataset

Open Dataset, choose upload, and select `examples/sft_sample.jsonl`. Confirm:

- detected format is `messages`;
- every row has valid user and assistant roles;
- examples match the intended educational behavior; and
- no evaluation prompt is duplicated in the training rows.

Add the inspected source to the dataset collection.

Step 3: inspect the model

Open Model and inspect `Qwen/Qwen3-0.6B`. Record the revision, parameter count if available, and the app's capacity warning. A model ID or repository-root HTTPS URL is accepted; arbitrary file URLs are not.

Step 4: save training settings

Choose SFT and QLoRA. Select Smoke and Auto compute. Use the standard backend for the first run so the exercise applies to Windows and Linux. Save the settings.

Step 5: review and launch

On Review & run, verify model, dataset, backend, effective compute type, method, length, 20-step cap, sample cap, learning rate, batching, and Hub upload disabled. Acknowledge a model-size warning only after understanding it.

Start training. The browser switches to Monitor while a child process runs. Additional experiments can be submitted from Review & run; they remain in a durable first-in-first-out queue.

Step 6: read the monitor

Observe state, message, the progress bar and whole-number percentage, metrics, and `training.log`. The percentage is a best-effort view of completed trainer steps, not an estimated time remaining. Identify:

- when model and dataset loading finish;
- the first reported training loss;
- whether all 20 steps run; and
- the final artifact directory.

If the run fails, diagnose the nearest boundary before changing random settings.

Step 7: compare outputs

After completion, run the five held-out prompts through base-versus-adapter comparison. Use the same rubric and deterministic prompt text. Record wins, ties, regressions, and unexpected style changes.

Step 8: inspect artifacts

Open `.runs/<run-id>/output/` and identify:

- `adapter/`;
- `metrics.json`;
- `training_config.json`; and
- any `checkpoint-*` directory.

Explain why `adapter_model.safetensors` is much smaller than the base model and why it cannot generate by itself.

Optional experiment: Unsloth

On native Windows with a ready runtime, repeat the same configuration with Unsloth. Do not compare quality from different random or data settings. Compare:

- successful model compatibility;
- elapsed time;
- peak VRAM;
- logs and artifact shape; and
- held-out output scores.

One small run cannot establish a general performance claim.

Lab report

Record configuration, environment, baseline table, final metrics, evaluation table, failures, and one next experiment. A screenshot of decreasing loss is not a lab report.

Module 13 - Lab B: preference training safely

Goal

Understand preference data and complete one small DPO pipeline. Reward, KTO, and ORPO variations demonstrate how the same source schema supports different objectives.

Important limitation

`examples/preference_sample.jsonl` is a tiny synthetic teaching file. It verifies the pipeline and illustrates clear preferences. It is not large or diverse enough for a production-quality alignment claim.

Step 1: inspect preference quality

For each row, ask:

- Is the chosen response clearly better under a written rubric?
- Is the rejected response plausible enough to provide learning signal?
- Does the pair differ in the behavior of interest rather than irrelevant wording?
- Are safety and factual claims defensible?

Write the rubric before training. Example dimensions are correctness, directness, operational safety, and honesty about limitations.

Step 2: create a preference baseline

Prepare five held-out prompts covering different rubric dimensions. Generate the base model's answer. If possible, create two candidate answers per prompt and label them blindly before seeing which system produced each.

Step 3: run DPO

Use:

Setting	Value
Model	`Qwen/Qwen3-0.6B`
Dataset	`examples/preference_sample.jsonl`
Approach	DPO Training
Method	QLoRA
Preset	Smoke test
Learning rate	Default `1e-5`
Beta	Default `0.1`
Compute	Auto

Inspect, add, save, review, and launch exactly as in Lab A. Confirm the dataset format is `preference`, not prompt/completion.

Step 4: evaluate DPO

Use held-out prompts and the same rubric. Pairwise blind comparison is more reliable than asking whether an output "looks better." Track preference win rate as:

```
win rate = adapter-preferred comparisons / decisive comparisons
```

Also record ties and regressions. With five examples, this is a diagnostic, not a statistically strong conclusion.

Variation A: Reward Modeling

Select Reward Modeling with default learning rate `1e-3`. The model path changes to a one-label sequence classifier. After training, inspect artifacts and metrics. The Monitor page will explain that generative comparison is unavailable. A complete reward evaluation tool would score held-out chosen/rejected pairs and measure ranking accuracy; that UI is not implemented.

Variation B: KTO

Select KTO and observe that the minimum per-device batch becomes 2. Keep accumulation and memory constraints in mind. Compare trainer behavior and metrics, but do not assume loss values are directly comparable across different objectives.

Variation C: ORPO

Select ORPO and keep the shared preference mapping. Review the objective distinction: ORPO combines supervised and preference pressure rather than reproducing DPO's exact reference-policy formulation.

Preference-data failure modes

- annotators use different rubrics;
- chosen answers are longer and length becomes an unintended shortcut;
- rejected answers are absurdly bad and teach little;
- pairs differ in factual content that reviewers did not verify;
- preference direction is accidentally reversed;
- demographic or cultural preferences are treated as universal; and
- training/evaluation pairs share templates or facts.

Lab report

Compare at least two objectives conceptually even if hardware time permits only one run. State what each artifact predicts, how it would be evaluated, and why trainer loss cannot be compared as a universal quality score.

Module 14 - Evaluate like an experimenter

Objectives

You will build evaluation sets, choose metrics, interpret curves, diagnose overfitting, and make defensible iteration decisions.

Evaluation starts with a product claim

Replace "make the model better" with a falsifiable claim such as:

> On held-out support questions, the adapter follows the required three-section format and avoids unsupported troubleshooting steps more often than the prompted base model.

The claim determines data, rubric, metrics, and reviewers.

Three evaluation layers

1. **Pipeline checks:** files load, shapes match, loss is finite, artifacts reopen.
2. **Task metrics:** exact match, classification accuracy, format compliance, ranking accuracy, or rubric scores.
3. **Behavioral review:** correctness, safety, usefulness, robustness, and regressions on realistic inputs.

Training, validation, and test

- Training data updates parameters.
- Validation data guides iteration and hyperparameters.
- Test data is reserved for the final estimate.

Repeatedly checking the test set and tuning to it turns it into validation data. Keep a final blind set or collect new cases before a release claim.

Read loss curves carefully

Observation	Possible interpretation	Next check
Training and validation loss decrease	Learning signal generalizes so far	Evaluate task outputs
Training decreases, validation worsens	Overfitting or distribution mismatch	Stop earlier, improve data, regularize
Both remain flat	Weak signal, wrong mapping, LR too low, frozen path issue	Inspect examples and gradients/logs
Loss spikes or becomes non-finite	LR/precision instability, bad values	Lower LR, inspect data and compute type
Loss decreases but outputs regress	Objective misaligned with product metric	Fix rubric/data, not just optimizer

Different objectives produce different loss scales. Do not compare SFT loss 1.2 and DPO loss 0.7 as if the smaller number identifies the better model.

Useful metrics

- **Perplexity** is `exp(cross_entropy)` for language modeling; lower means targets are less surprising, not necessarily more useful.
- **Format compliance** checks required structure programmatically.
- **Pairwise win rate** records blinded human or judge preferences.

- **Reward ranking accuracy** checks whether chosen items score above rejected items.
- **Safety violation rate** counts rubric-defined failures.
- **Latency and memory** measure deployment feasibility.

Automated LLM judges can scale review but introduce model bias, prompt sensitivity, and correlated errors. Calibrate them against human labels.

Hyperparameter strategy

Use a small, fixed evaluation suite and change one variable. Recommended search order:

1. verify labels and formatting;
2. improve coverage and remove contradictions;
3. tune learning rate by a small multiplicative grid;
4. adjust epochs/steps based on validation behavior;
5. adjust length only for truncation evidence;
6. adjust effective batch for stability; and
7. compare adapter methods/backends after the recipe is stable.

Reproducibility record

Store:

- model ID and immutable revision when possible;
- dataset IDs, configurations, splits, and file hashes;
- exact `training_config.json`;
- package lockfiles, GPU, driver, and CUDA information;
- seed and evaluation prompts;
- artifact checksums; and
- reviewer rubric and raw scores.

The same seed does not guarantee bit-identical GPU results across drivers, kernels, hardware, or package versions.

Exercise: design an evaluation card

Write one page containing task claim, excluded claims, baseline, sample source, split policy, metrics, rubric, reviewer process, minimum improvement threshold, safety gates, and known limitations.

Checkpoint

1. Why can validation loss improve while product quality gets worse?
2. Why must objective losses not be compared across trainers?
3. What turns a test set into a validation set?

Module 15 - Operate jobs and use artifacts

Objectives

You will understand process isolation, status files, cancellation, recovery, adapter inference, Hub publishing, and Ollama's separate role.

Why training runs in another process

Streamlit reruns scripts after UI interactions. Long GPU training inside that lifecycle would be fragile. The app instead serializes `TrainingConfig` and starts:

```
python -m lora_finetune_studio.worker <absolute-config-path>
```

The main app remains responsive while the worker owns model memory. Standard jobs use the main interpreter; Unsloth jobs use `.venv-unsloth` with repository `src/` added to `PYTHONPATH`.

Durable run layout

```
.runs/<run-id>/
|-- config.json
|-- status.json
|-- training.log
`-- output/
    |-- checkpoint-*/
    |-- adapter/
    |-- metrics.json
    `-- training_config.json
```

Status JSON is written to a temporary file and atomically replaced, so the two-second Monitor poll does not read half-written state.

State machine

```
queued -> running -> completed -> dispatch next queued run
                  -> failed ----> dispatch next queued run
                  -> cancelled -> dispatch next queued run

failed/cancelled -> queue newest checkpoint -> queued
```

Only one owned training PID can be active. Before cancellation, the app verifies that the stored PID command line is the expected worker with the expected config path. It terminates, waits ten seconds, and kills only after timeout.

Queue order is stored in `.runs/queue.json` under an OS-level file lock. After a terminal status, a lightweight handoff waits for the old worker to exit and release VRAM before launching the next run. The queue survives app restarts and continues after failures. Confirmed app shutdown cancels the active worker without starting another; waiting runs resume when the app starts again.

Resume chooses the numerically latest `checkpoint-*`, writes its absolute path into the existing config, and appends the same run ID to the queue. Resume cannot recover work done before the first saved checkpoint.

Logs and error handling

Worker stdout/stderr append to `training.log`. The status stores a short error with the exact HF token redacted; the log receives the traceback. Treat logs as potentially sensitive because third-party diagnostics can contain paths, repository names, or data details.

Base-versus-adapter comparison

For generative approaches, the app loads the base in four-bit NF4, generates deterministically, releases memory, then repeats with `PeftModel` attached. Sequential loading avoids holding two base models simultaneously. The comparison is qualitative; use Module 14 for a proper evaluation suite.

Publishing to Hugging Face

When enabled, a successful run pushes trainer model/adapter state and tokenizer to the configured repository using `HF_TOKEN`. It does not publish a merged full model. Review the destination, license, data rights, model card, and access level before upload.

Ollama boundary

The Ollama page calls local `/api/tags` and `/api/generate` endpoints for models already installed in Ollama. It does not convert PEFT adapters, merge them, build GGUF files, or create an Ollama model. Those are separate deployment workflows.

GPU memory cleanup

The GPU page reports global free/total memory and main-process PyTorch allocated/reserved memory. Garbage collection plus `torch.cuda.empty_cache()` releases unused cache blocks, not live tensors or allocations owned by the worker, Ollama, or another application. Cleanup is disabled during active training.

Checkpoint

1. Why does closing the browser not necessarily stop training?
2. What does atomic status replacement protect against?
3. Why does the adapter comparison load models sequentially?

Module 16 - Read and extend the repository

Objectives

You will trace configuration from UI to trainer, understand security boundaries, and make changes at the narrowest responsible layer.

Runtime architecture

```
Browser
  -> streamlit_app.py and app_pages/
  -> models.py contracts + sources.py/hardware.py boundaries
  -> jobs.py process manager
  -> worker.py
  -> training.py
  -> Hugging Face libraries and .runs artifacts
```

`streamlit_app.py` initializes shared per-session state and eight `st.Page` entries. Page files own UI, while reusable and testable behavior lives in the package.

The public showcase is a separate **Implemented** entry point: `demo/streamlit_app.py` reads `demo/fixtures/showcase.json` and never starts `jobs.py`, `worker.py`, or a CUDA stack. Tests confirm that fixture still satisfies `TrainingConfig` and `JobStatus`.

Contract-first flow

`TrainingConfig` is the central process contract. UI values are converted to enums and dataclasses, validated, serialized to JSON, reconstructed by the worker, and mapped to TRL arguments. Defaults in `from_dict` preserve old run compatibility.

When adding a setting:

1. add a JSON-serializable field and default;
2. add legacy deserialization behavior when necessary;
3. validate it in the shared contract;
4. expose it in Training and Review;
5. pass it at the model/trainer boundary; and
6. test serialization, validation, and behavior.

Dataset boundary

`sources.py` validates repository roots, uploads, loading, and inspection. `training.py` normalizes the saved mapping. Add a new shape to both sides and test the round trip; otherwise the UI can accept data the worker cannot consume.

Trainer boundary

`training.py` resolves quantization, PEFT configuration, model type, TRL trainer, and trainer arguments. A new approach requires more than adding a dropdown option: define data, validation, model class, loss/trainer, controls, artifacts, evaluation, errors, tests, and documentation.

Job boundary

`jobs.py` owns `.runs`, atomic writes, one-job detection, process creation, verified cancellation, checkpoint selection, and log tails. Preserve token-free config, PID ownership, run-ID validation, and atomic visibility when changing orchestration.

Security controls

- Hugging Face URLs must target repository roots on the expected HTTPS host.
- Uploaded content is extension/size checked and stored by hash.
- Run IDs allow lowercase letters, digits, and hyphens only.
- Model/tokenizer loading disables remote repository code.
- Standard model loading requires safetensors.
- Tokens remain outside serialized training contracts.
- Cancellation verifies the worker command and config path.

These controls do not make untrusted datasets, model files, checkpoints, packages, or logs harmless. Use normal supply-chain, privacy, and access-control practices.

Testing layers

Tests	Boundary
`test_models.py`	contracts, migration, validation, presets, run paths
`test_sources.py`	URLs, uploads, inspection, loading
`test_training.py`	normalization, splits, adapters, quantization, trainers
`test_jobs.py`	atomic files, ownership, lifecycle, resume
`test_hardware.py`	scans, capacity guidance, allocator behavior
`test_inference.py`	adapter loading, generation, cleanup
`test_app.py`	Streamlit startup and UI contracts
`test_demo.py`	showcase fixture contracts and read-only startup
`test_tutorial.py`	handbook completeness and portable generated-asset hashes

CI runs formatting, linting, type checking, tests, and `scripts/build_tutorial.py --check` on Windows and Ubuntu. The check compares handbook text after newline normalization and PDF metadata, page size, and extracted text. GPU model loading and full training need an additional hardware smoke test.

Maintainer exercise

Trace `learning_rate` from its recipe default through Streamlit session state, the saved `TrainingConfig`, `config.json`, worker deserialization, TRL config, and `training_config.json`. Repeat for `use_unsloth` and identify where the interpreter changes.

Checkpoint

1. Why is `TrainingConfig` the central boundary?
2. Which two layers must change for a new dataset shape?
3. Why is an installed TRL trainer not sufficient evidence of app support?

Module 17 - Capstone: train a defensible specialist

Goal

Design, execute, and report one complete adapter experiment. Mastery means making defensible decisions and recognizing uncertainty, not merely finishing a GPU run.

Deliverable 1: task charter

Write:

- user and workflow;
- input and desired output;
- measurable success claim;
- constraints and prohibited behavior;
- why prompting or RAG is insufficient;
- deployment model and hardware assumptions; and
- stop conditions for safety, cost, or quality.

Deliverable 2: baseline and evaluation

Create at least 30 held-out cases spanning ordinary, difficult, adversarial, and out-of-scope inputs. Define a rubric before observing fine-tuned outputs. Score the base model with the best reasonable prompt.

Deliverable 3: data card

Document source, authorization, collection method, transformations, schema, row count, language/domain coverage, deduplication, leakage controls, privacy review, known biases, and excluded uses.

Start with a small reviewed dataset. Every row should have a reason to exist.

Deliverable 4: experiment plan

Choose approach, method, backend, model, revision, preset, and custom settings. For each, state a hypothesis. Define the first Smoke run and no more than three follow-up runs, each changing one primary variable.

Deliverable 5: execution record

Save environment details, run IDs, configs, logs, metrics, artifact checksums, failures, and deviations from the plan. A failed run with a correct diagnosis is valid evidence.

Deliverable 6: evaluation report

Compare base and adapter blind where possible. Report per-category results, confidence limits or sample-size caveats, regressions, safety failures, latency/memory, and examples that explain aggregate scores.

Deliverable 7: release decision

Choose one:

- **No-go:** evidence does not beat the baseline or risks are unacceptable.
- **Iterate:** a specific data or configuration change is justified by failures.

- **Pilot:** limited users and monitoring with explicit rollback conditions.
- **Release:** evaluation and operations meet predefined gates.

Do not choose release because training completed or because one output looks impressive.

Mastery rubric

Capability	Evidence
Explain	You can teach tokens, attention, loss, adapters, and objectives accurately
Design	Technique follows from a measured task and constraints
Build	Data passes structural and human quality review
Operate	Runs are reproducible, monitored, and recoverable
Evaluate	Held-out metrics and rubrics support the claim
Diagnose	Failures map to data, model, memory, objective, or orchestration boundaries
Govern	Rights, privacy, safety, and limitations are documented
Extend	Code changes preserve shared contracts and tests

Appendix A - Checkpoint answers

Module 1

1. It contains only learned adapter parameters and configuration; base weights remain external.
2. SFT/Reward/DPO/KTO/ORPO are objectives; LoRA/QLoRA/OFT/QOFT are methods.
3. Evaluation defines success and prevents optimizing only for training loss.

Module 2

1. Tokens are vocabulary units and can be smaller or larger than words.
2. The language model already embeds token IDs; a separate embedding model is needed for retrieval/vector-search workflows, not this trainer.
3. Roles or separators can be misinterpreted, targets can be trained incorrectly, and inference formatting can mismatch training.

Module 3

1. A step updates parameters; an epoch passes through the dataset.
2. It processes microbatches sequentially and updates only after gradients accumulate.
3. Activations, adapter gradients, optimizer state, kernels, and other processes still consume memory.

Module 4

1. Without it, a training position could copy future target tokens.
2. Queries seek, keys advertise, and values carry the mixed information.
3. They contain the block's learned transformations and can be adapted efficiently.

Module 5

1. When answers depend on current/private attributable knowledge rather than stable behavior.
2. SFT supplies targets; preference data supplies relative judgments.
3. It requires broad corpora and compute far beyond the local post-training pipeline.

Module 6

1. Base weights, activations, kernels, adapter gradients, and optimizer state.
2. The frozen base is unchanged; each task stores only its adapter.
3. Method controls trainable parameterization; objective controls the loss/data meaning.

Module 7

1. Adapter capacity and trainable parameter count.
2. Frozen base weights are four-bit; adapter weights train with floating-point compute.
3. Vector norms and angles.

Module 8

1. Concatenation and the selected trainer require one compatible schema.

2 No; contribution follows row count.

3 Random splits can leak related examples and do not represent a final external test.

Module 9

1 It produces scalar scores rather than a causal text-generation policy.

2 DPO uses offline pairs directly; PPO requires rollouts and additional models/state.

3 Preference pressure relative to the reference/objective formulation.

Module 10

1 Its pinned Python/PyTorch stack differs from the main app.

2 No; it only authenticates repository access.

3 Real memory depends on architecture, settings, kernels, fragmentation, and processes.

Module 11

1 The explicit 20-step cap.

2 It produces a deterministic representative subset rather than always taking the first rows.

3 Sequence length and per-device batch size.

Module 14

1 The optimized likelihood can diverge from product behavior or overfit.

2 Their mathematical scales and meanings differ.

3 Repeated tuning based on its results.

Module 15

1 The worker is a separate operating-system process.

2 Readers cannot observe partially written JSON.

3 It lowers simultaneous VRAM use.

Module 16

1 It is the serialized UI-to-worker contract.

2 Inspection/mapping and worker normalization.

3 App support also requires data, validation, model, controls, artifacts, tests, and UI.

Appendix B - Formula and settings reference

Concept	Formula or rule
Next-token factorization	`P(x_1..x_T) = product_t P(x_t | x_<t)`
Softmax	`exp(z_i) / sum_j exp(z_j)`
Cross-entropy for target y	`-log P(y | context)`
Gradient update	`theta <- theta - learning_rate * gradient`
Effective batch	`batch_size * accumulation * devices`
Attention	`softmax(Q K^T / sqrt(d_k) + mask) V`
LoRA update	`W' = W + (alpha/r) B A`
LoRA parameter count	`r * (d_in + d_out)` per adapted dense matrix
Orthogonality	`R^T R = I`
Perplexity	`exp(mean cross-entropy)`
Pairwise reward preference	Encourage `r(chosen) > r(rejected)`

App setting	Contract
Maximum length	128-8192
Epochs	Greater than 0, at most 20
Learning rate	`1e-7` through `1e-2`
Maximum samples	Positive integer or all
Maximum gradient norm	Finite and non-negative
Beta	Positive for DPO/KTO/ORPO
KTO batch	At least 2
Unsloth	Windows, LoRA/QLoRA, non-FP32
Evaluation split	Skipped when disabled or fewer than 10 combined rows

Appendix C - Troubleshooting by boundary

Symptom	Boundary	First evidence and action
Launcher fails	uv/runtime/port	Read `.runs/streamlit.err.log`; verify network and port 8504
CUDA unavailable	driver/PyTorch	Check System and `torch.cuda.is_available()`
Token 401/403	Hub access	Verify token scope and gated-model acceptance
Dataset inspection fails	URL/file/schema	Check root URL, subset, split, extension, and columns
Review blocks start	config/readiness	Read every displayed blocker; do not bypass validation
Out of memory	model/length/batch/processes	Use smaller model, QLoRA, shorter length, batch 1; close GPU apps
Non-finite loss	data/LR/precision	Inspect values, lower LR, compare compute type
Unsloth disabled	runtime/method/compute	Use native Windows, LoRA/QLoRA, BF16/FP16, prepared runtime
Worker failed	model/data/trainer	Read failed status and the end of `training.log`
Resume unavailable	checkpoints	The run stopped before a checkpoint; start a new run
Comparison fails	revision/VRAM/adapter	Match base revision and inspect Monitor/server logs
Ollama empty	optional service	Start Ollama and pull a model; training is independent

Appendix D - Glossary

Activation: Intermediate tensor produced during a forward pass.

Adapter: Small set of learned parameters attached to a frozen base model.

Attention: Content-dependent weighted mixing of token representations.

Autoregressive: Generates each new token conditioned on previous tokens.

Backpropagation: Chain-rule computation of loss gradients.

Base model: Pretrained model onto which an adapter is attached.

Batch: Examples processed together before or during one update.

BF16: 16-bit floating format with wide exponent range.

Causal mask: Prevents a token from attending to future tokens.

Checkpoint: Intermediate state used to resume training.

Corpus: Collection of language data.

CUDA: NVIDIA's GPU computing platform used by PyTorch here.

Epoch: One pass through the training dataset.

Evaluation set: Examples excluded from parameter updates and used to measure behavior.

FP16/FP32: 16-bit and 32-bit floating-point formats.

Gradient: Derivative of loss with respect to a parameter.

Gradient accumulation: Adds microbatch gradients before an optimizer step.

Gradient checkpointing: Recomputes activations during backward to save memory.

Hub: Hugging Face service for model, dataset, and adapter repositories.

Inference: Using learned parameters to generate or score outputs.

Learning rate: Scale applied to optimizer updates.

Logit: Unnormalized score before softmax.

Loss: Scalar objective minimized during training.

NF4: Four-bit quantization type designed for normally distributed weights.

Optimizer: Algorithm that updates trainable parameters from gradients.

Overfitting: Improving on training data while generalization worsens.

Parameter: Learned numeric value in a model.

PEFT: Parameter-efficient fine-tuning.

Quantization: Lower-precision representation of weights or activations.

Rank: Inner dimension controlling LoRA update capacity.

Reference model: Fixed policy used by some preference objectives.

Residual stream: Hidden representation refined across transformer blocks.

RAG: Retrieval-augmented generation using external retrieved context.

Safetensors: Tensor storage format designed to avoid executable pickle loading.

Seed: Initial value controlling pseudorandom operations.

Softmax: Converts logits into normalized positive probabilities.

Token: Vocabulary unit consumed or generated by a model.

Tokenizer: Maps text to token IDs and back.

Trainer: Library component coordinating data, forward/backward passes, evaluation, checkpointing, and metrics.

Transformer: Neural architecture based on attention and repeated blocks.

TRL: Hugging Face library providing post-training trainers.

VRAM: GPU memory used for weights, activations, gradients, state, and kernels.

Appendix E - Official references

The repository implementation and tests define app behavior. These official sources provide broader theory and library details:

- Hugging Face LLM Course
- Transformers documentation
- Datasets documentation
- PEFT documentation
- PEFT LoRA guide
- TRL documentation
- TRL PEFT integration
- bitsandbytes documentation
- PyTorch documentation
- Streamlit documentation
- Unsloth documentation
- uv project documentation
- Hugging Face Hub documentation
- Ollama API documentation

Further foundational papers

- Vaswani et al., Attention Is All You Need
- Hu et al., LoRA: Low-Rank Adaptation of Large Language Models
- Dettmers et al., QLoRA: Efficient Finetuning of Quantized LLMs
- Qiu et al., Controlling Text-to-Image Diffusion by Orthogonal Finetuning
- Ouyang et al., Training language models to follow instructions with human feedback
- Rafailov et al., Direct Preference Optimization